Guide du projet — SMS-GATEWAY

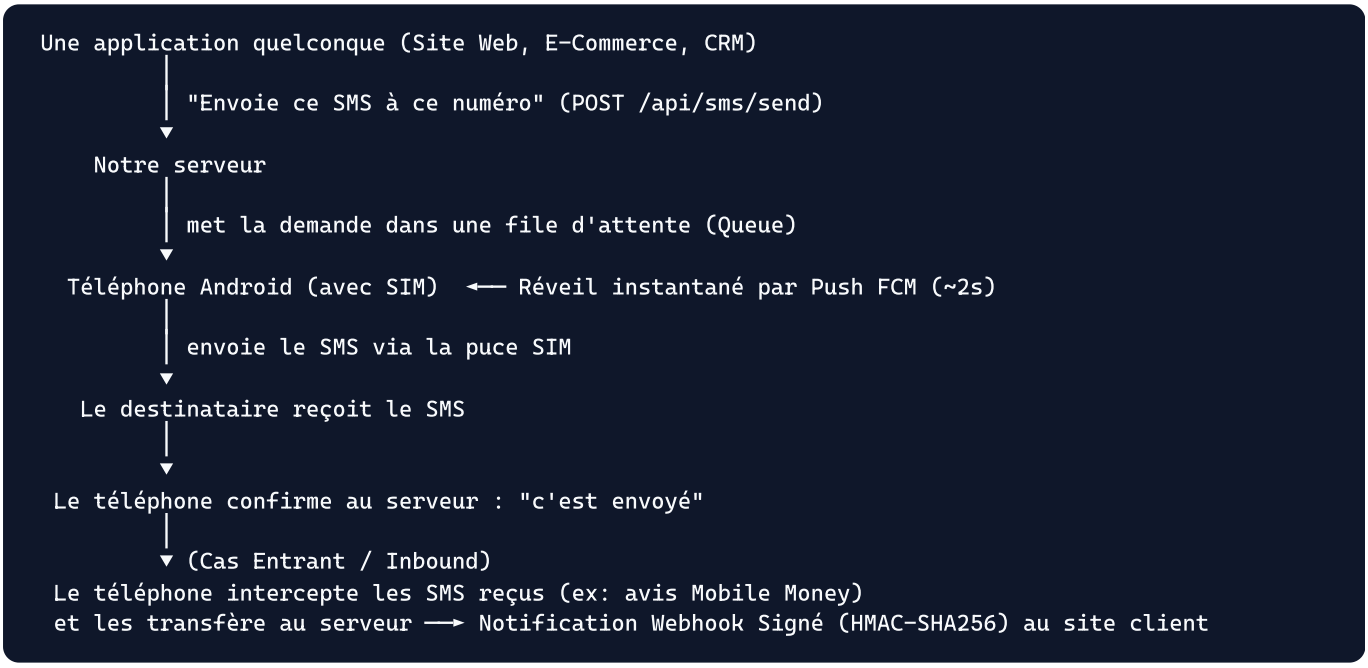
Document de conception & spécifications techniques à jour (Version Production)

1. C'est quoi ce projet, en simple ?

On veut construire un système qui permet à n'importe quelle application (un site web, un logiciel de gestion, une app métier...) d'envoyer et de recevoir des SMS **sans passer par un service payant type Twilio ou une API SMS externe**.

À la place, le système utilise un **vrai téléphone Android avec une vraie carte SIM** pour envoyer et recevoir les SMS. Ce téléphone doit juste être **connecté à internet** quelque part (au bureau par exemple) avec l'app installée qui tourne **en arrière-plan** — l'écran peut être éteint, personne n'a besoin de le tenir en main. C'est justement le rôle de l'app : attendre en permanence, sans surveillance humaine, qu'une nouvelle demande d'envoi arrive ou qu'un SMS soit reçu.

Le principe en une image (Flux bidirectionnel) :



Tu vas donc construire **deux choses principales** :

1. **Un serveur** (le cerveau : reçoit les demandes, gère la file d'attente, garde l'historique, gère la sécurité et les Webhooks).
2. **Une app Android** (les bras : reçoit les ordres du serveur, envoie réellement les SMS, intercepte les SMS entrants).

2. Le vocabulaire à connaître avant de commencer

Terme	Explication simple
API	Une porte d'entrée pour que d'autres programmes puissent parler à ton serveur (ex: `POST /api/sms/send`).
File d'attente (queue)	Une liste des SMS à envoyer, dans l'ordre, pour ne pas tout envoyer en même temps.
Notification push	Un message envoyé au téléphone pour le réveiller et lui dire "tu as une nouvelle tâche", même si l'app est fermée.
Foreground Service	Un mode spécial sur Android qui permet à une app de tourner en arrière-plan sans être arrêtée par le système (comme quand Spotify joue de la musique en fond).
Statut d'un SMS	Où en est un SMS dans son parcours : en attente, réclamé, en cours d'envoi, envoyé, livré, échoué.
Token / Authentification	Une clé secrète qui prouve qu'un appareil ou un utilisateur a le droit de faire une action.
Idempotence	Le fait qu'une même demande envoyée deux fois par erreur ne doit provoquer qu'un seul envoi réel.
Inbound / SMS Entrant	Capacité de l'app Android à capturer les SMS arrivant sur la SIM et les transmettre au serveur.
Webhook Signé (HMAC)	Notification HTTP automatique vers le site du client lors de la réception d'un SMS, signée cryptographiquement avec `HMAC-SHA256` pour garantir la sécurité.
Accusé Réseau (DLR)	Confirmation au niveau de l'antenne relais opérateur (`LIVRE` / `ECHEC_LIVRAISON`) via `deliveryIntent`.
Multi-SIM & Rotation	Gestion de plusieurs SIMs avec bascule automatique tous les 10 SMS pour équilibrer la charge.
Code USSD	Commande réseau (ex: `*111#`) exécutée automatiquement par l'app pour relever le solde de la SIM.

3. Les deux briques à construire

A. Le serveur

Le serveur doit savoir répondre à ces questions à tout moment :

- Quels appareils (téléphones) sont enregistrés, et sont-ils actuellement disponibles ?
- Quels SMS sont en attente d'envoi ?
- Quel SMS a été confié à quel appareil ?
- Quel est le résultat final de chaque SMS ?

Statuts d'un appareil :

```
ONLINE / EN_LIGNE ⟶ le téléphone est connecté, actif et disponible
OFFLINE / HORS_LIGNE ⟶ le téléphone n'est pas connecté ou service arrêté
BUSY ⟶ le téléphone est en train d'envoyer un SMS
DISABLED / DESACTIVE ⟶ le téléphone a été désactivé manuellement
```

Statuts d'un SMS (Cycle de vie complet) :

```
PENDING / EN_ATTENTE ⟶ en attente d'être pris en charge
ASSIGNED / RECLAME ⟶ confié/réclamé par un appareil précis (anti-doublon)
SENDING ⟶ en cours d'envoi par le modem radio GSM
SENT / ENVOYE ⟶ envoyé avec succès à la radio GSM
DELIVERED / LIVRE ⟶ confirmé délivré sur le mobile destinataire (DLR réseau)
FAILED / ECHOUE ⟶ échec de l'envoi (crédit insuffisant, pas de réseau)
```

B. L'application Android

Elle doit, dans l'ordre :

1. Se connecter au serveur et prouver son identité (authentification Firebase / QR Code).
2. Rester "éveillée" même quand on ne l'utilise pas (Foreground Service).
3. Recevoir une notification push (FCM) quand une nouvelle tâche arrive.
4. Aller chercher les détails de cette tâche auprès du serveur.
5. Envoyer le SMS via la carte SIM du téléphone (avec choix de la SIM ou auto-rotation).
6. Dire au serveur si ça a marché ou pas (confirmation avec rejeu en cas de coupure).
7. Capturer les SMS reçus sur la SIM et les transmettre au serveur (Mode Inbound).
8. Savoir reprendre normalement après une coupure internet ou une panne.

Elle doit aussi avoir un écran simple pour voir : l'état du téléphone, le réseau utilisé, la batterie, les SMS envoyés/en attente, le journal des événements.

4. Les 4 pièges techniques à connaître avant de coder

Piège 1 — Ton app va s'arrêter toute seule si tu ne fais rien de spécial

Android essaie d'économiser la batterie en tuant les apps qui tournent en arrière-plan. Sur certaines marques (Xiaomi, Huawei, Oppo), c'est encore plus agressif.

Solution : utiliser un **Foreground Service** (avec une notification permanente visible `SMSTSIKA actif`) + **WorkManager/FCM**. C'est la toute première chose à valider techniquement, avant même de construire le reste — si l'app meurt en arrière-plan, rien d'autre ne fonctionnera. Ajouter également un récepteur de démarrage (`BOOT_COMPLETED`) pour relancer l'app si le téléphone redémarre.

Piège 2 — Une carte SIM ne peut pas envoyer des SMS à l'infini

Les opérateurs bloquent une SIM qui envoie trop de SMS trop vite (souvent au-delà de 20 à 100/heure, selon l'opérateur).

Solution : c'est pour ça qu'on gère plusieurs téléphones en parallèle et le multi-SIM. Répartir les tâches entre appareils avec une limite par heure et par SIM, temporiser les envois (Pacing), et effectuer une rotation automatique des SIMs tous les 10 SMS.

Piège 3 — Attention au double envoi

Si le téléphone envoie le SMS mais perd internet juste après, le serveur peut croire que ça a échoué et redemander l'envoi. Résultat : la personne reçoit le SMS deux fois.

Solution : chaque tâche a un identifiant unique, un statut intermédiaire `RECLAME` (ASSIGNED), et un délai d'attente avant de considérer que ça a réellement échoué. Le SMS n'est émis qu'une seule fois ; si le réseau coupe après l'envoi, seule la confirmation HTTP est mise en file locale et rejouée dès la reconnexion (Idempotence).

Piège 4 — L'app ne pourra pas aller sur le Google Play Store

Depuis 2019, Google interdit l'usage de l'API SMS pour la plupart des apps du Play Store. Ce n'est pas un bug de ton code, c'est une règle de Google.

Solution : l'app sera installée manuellement (fichier APK) sur chaque téléphone dédié, téléchargée depuis le serveur ou le QR code. C'est normal pour ce type de projet interne.

5. Plan sur 3 mois, étape par étape

Mois 1 — Prouver que ça marche

Objectif : un seul téléphone reçoit un ordre et envoie un vrai SMS.

- **Semaine 1-2 :** comprendre les contraintes Android (piège 1), poser l'architecture générale — créer un petit serveur (API basique) + une app Android qui s'authentifie.
- **Semaine 3-4 :** premier envoi de SMS fonctionnel de bout en bout, y compris avec l'app en arrière-plan (test sur vrai téléphone, pas seulement émulateur).

Mois 2 — Construire le système complet

- File d'attente des SMS (Queue) et gestion de la réception (Inbound).
- Notifications push (Firebase Cloud Messaging) pour réveil instantané (~2s).
- Gestion de plusieurs appareils en parallèle + stratégie de répartition (piège 2) + Multi-SIM.
- API complète pour les applications externes avec clés API et Webhooks signés HMAC-SHA256.
- Dashboard web (statistiques, historique, gestion des appareils, graphique de consommation quotidienne).

Mois 3 — Rendre le système fiable et propre

- Sécurité : authentification, tokens, HTTPS, limitation des requêtes (RLS Supabase).
- Gestion de toutes les erreurs possibles (réseau coupé, SIM indisponible, accusés DLR, solde USSD, etc.).
- Tests spécifiques anti-doublon (piège 3) : couper le wifi volontairement pour vérifier l'idempotence.
- Documentation technique + procédure d'installation.
- Préparer la démonstration finale.

6. Livrables attendus à la fin du stage

- Le serveur central qui fonctionne (Next.js + Supabase + Firebase).
- L'API d'envoi de SMS et de réception Webhook, documentée et sécurisée.
- L'application Android installable (APK) avec Foreground Service, Multi-SIM, DLR et USSD.
- Le système de notifications push (FCM) pour réveil instantané.
- La gestion de plusieurs téléphones et cartes SIM en même temps.
- Le dashboard web client avec graphique de consommation quotidienne et gestion des quotas.
- La documentation technique (comment c'est construit) et d'installation (comment le déployer).
- Une démonstration fonctionnelle globale.

7. Conseil général

Le choix des technologies (langages, frameworks, base de données) est libre. Mais à chaque choix, il faut pouvoir répondre à trois questions dans le rapport final : **pourquoi ce choix, quelles étaient les alternatives, quels sont les avantages/inconvénients**. Pas besoin de connaître la réponse parfaite dès le départ — l'important est de documenter le raisonnement au fur et à mesure, pas de le reconstruire à la fin.